

Strutture dati:

Una struttura dati è un insieme di dati che vengono aggregati. Se i dati aggregati sono dello stesso tipo si parla di array, se sono diversi di record. Le strutture dati sono dette ADT (Abstract Data Type), siccome le loro funzioni sono le stesse per ogni tipo di dato per cui vengono usate.

Strutture statiche e dinamiche:

Gli array sono strutture dati di tipo statico, in fase di creazione viene definita la dimensione del vettore, le celle di memoria vengono allocate immediatamente ed in modo contiguo. Il tempo di accesso ai dati molto ridotto ma talvolta si occupa memoria inutilmente in quanto mediamente non si arriverà ad occupare l'intero spazio. Inoltre, nel caso in cui si debbano inserire nell'array più elementi del previsto non sarà possibile.

Le liste sono strutture dati di tipo dinamico, quando vengono create non allocano memoria fin da subito, ma solo quando viene richiesta, se non più necessaria a contenere un elemento la memoria viene deallocata. Quando si alloca spazio nella memoria, non necessariamente le celle sono contigue; questo causa un allungamento del tempo di accesso. Dato che non c'è contiguità tra i dati inseriti, ogni elemento della lista ha un riferimento all'elemento successivo, che permette quindi un accesso sequenziale a tutti i dati.

Lista dinamica semplice:

Nella lista dinamica semplice, a differenza di un array, il puntatore (head L) è il primo elemento inserito all'interno della lista. Si può accedere. Da qui, per ogni elemento è possibile accedere al successivo tramite il campo, di tipo puntatore, che contiene il riferimento all'elemento successivo. È possibile eliminare dati dalla lista, ma dopo la cancellazione di un elemento è necessario ricollegare il dell'elemento precedente all'elemento successivo.

Lista dinamica doppia:

Questa lista aggiunge un puntatore prev che tiene traccia dell'elemento successivo, in aggiunta al puntatore next già presente nella lista dinamica semplice.

Operazioni sulle liste:

Si utilizzano una lettera maiuscola come D o S per indicare la struttura dati, x per indicare un elemento e k per indicare una chiave, le operazioni sono le seguenti:

- Insert(S, x): inserisce un elemento all'interno della lista.
- Delete(S, x): cancella un elemento all'interno della lista.
- Search(S, k): cerca un valore all'interno della lista.
- Update(S, x): aggiorna un elemento specifico all'interno di una lista (delete seguito da un insert)
- Min(S): trova il minimo all'interno della lista.
- Max(S): trova il max all'interno della lista.
- Pred(S, k): trova il predecessore.
- Succ(S, k): trova il successore.
- Free(S,x): libera la posizione ma non la memoria

Stack (pile):

Uno stack è una struttura dati dove i dati al suo interno vengono gestiti tramite una politica LIFO (Last In First Out).

Operazione	input	output	risultato
Push	S, K	Void	Inserisce k in cima allo stack
Pop	S	K	Rimuove e restituisce dallo stack l'ultimo elemento inserito
StackEmpty	S	Boolean	Restituisce se lo stack è vuoto
Top	S	K	Restituisce il valore in cima allo stack senza rimuoverlo

Queue (code):

Le code sono strutture dati che gestiscono i dati secondo una politica FIFO (First In First Out).

Operazione	input	output	risultato
Enqueue	S, K	Void	Inserisce k in cima alla queue
Dequeue	S	K	Rimuove e restituisce dalla queue l'ultimo elemento inserito
QueueEmpty	S	Boolean	Restituisce se la queue è vuota

Alberi:

Un albero è un grafo non orientato, connesso e aciclico, ciò significa che i cammini che collegano i vertici (o nodi) del grafo sono percorribili in entrambe le vie, ogni nodo è raggiungibile tramite uno o più cammini; tuttavia, non è possibile eseguire un ciclo.

Ogni albero ha una radice (root) che rappresenta il punto di partenza dell'albero, da cui tutti i nodi partono. Ogni nodo (eccetto la radice) ha degli antenati, nodi che lo precedono nel cammino dalla radice al nodo, e dei discendenti, nodi che si trovano nella posizione successiva a quella del nodo in questione nel cammino. I nodi che non hanno figli sono detti foglie. La profondità di un nodo è la lunghezza del cammino dalla radice alla posizione in cui si trova. l'altezza dell'albero invece è la massima profondità ottenibile, ovvero il più lungo cammino dalla radice alla foglia.

Proprietà generali dell'albero:

- Completo: ogni nodo ha due figli e tutti i nodi fino ad un determinato livello hanno figli.
- Semi-completo: è un albero completo per tutti i livelli eccetto l'ultimo, il numero di nodi totali è: $\sum_{i=0}^n 2^i = 2^{n+1} - 1$

Delete on bst:

1. Eliminare una foglia: rimuovo la foglia e pongo il figlio del predecessore a null.
2. Eliminare un nodo che ha un solo figlio: collego il parent del nodo che voglio eliminare al figlio dello stesso.
3. Eliminare nodi che hanno più figli:
 - a. Trovo il minimo nel sottoalbero destro, copio il valore del minimo nella posizione del nodo che voglio sostituire, cancello il duplicato del sottoalbero destro.
 - b. Trovo il massimo nel sottoalbero sinistro, copio il valore del massimo nella posizione del nodo che voglio sostituire, cancello il duplicato del sottoalbero sinistro.

Grafi

Rappresentazione dei grafi in memoria:

1. **Liste di adiacenza:** la lista di adiacenza ha n elementi fissi più tutti gli elementi collegati, può quindi contenere al massimo $n * m$ elementi dove m rappresenta la quantità di lati del grafo,

$n + m$ invece è la quantità effettiva di dati collegati. La lista di adiacenza è utilizzata per grafi sparsi, un grafo sparso soddisfa la seguente proprietà: $|E| \ll |V|^2$.

2. **Matrice di adiacenza:** la matrice di adiacenza è una matrice quadrata $n \times m$ che viene utilizzata quando vogliamo risparmiare tempo pagando in termini di spazio, in particolare quando il grafo è denso.

Visita in ampiezza (breadth-Frist-Search):

Dato un grafo $G = (V, E)$ ed un vertice s in V la ricerca in ampiezza visita sistematicamente, partendo dalla sorgente s , il grafo per scoprire tutti i vertici che sono raggiungibili da s . Nel contempo calcola la distanza di ogni vertice del grafo dalla sorgente s (lunghezza minima di un cammino dalla sorgente al vertice). Esso produce anche l'albero della ricerca in ampiezza i cui rami sono cammini di lunghezza minima. La ricerca viene detta in ampiezza perché l'algoritmo espande uniformemente la frontiera tra i vertici scoperti e quelli non ancora scoperti. In altre parole, scopre tutti i vertici a distanza k da s prima di scoprire quelli a distanza $k+1$.

```
BFS(G, s)
for "ogni vertice v ∈ V[G]" do
  color[v] ← bianco, d[v] ← ∞, p[v] ← nil
color[s] ← grigio, d[s] ← 0
Enqueue(Q, s)
while not Empty(Q) do
  u=Dequeue(Q)
  color[u] ← nero
  for "ogni v ∈ Adj[u]" do
    if color[v] = bianco then
      color[v] ← grigio, d[v] ← d[u] + 1, p[v] ← u
  Enqueue(Q,v)
```

Visita in profondità (Depth-first-Search):

La DFS esplora il grafo andando ogni volta il più possibile in profondità, per ogni vertice v in visita si prosegue la visita sugli archi non ancora esplorati. Se sul vertice v in visita si sono esplorati tutti gli archi si torna al vertice di origine (padre). Se sul grafo rimane qualche vertice non scoperto si ricomincia la visita in profondità su uno di quei vertici, l'intero processo è ripetuto finché non vengono scoperti tutti i vertici del grafo.

```
DFS(G)
for ogni vertice u in V[G] // inizializzazione di ogni vertice
do color[u] ← WHITE
p[u] ← NIL
time ← 0
for ogni vertice u in V[G]
do if color[u] = WHITE
then DFS-VISIT(G, u) // visita da ogni vertice non ancora scoperto
```

Heap:

Lo heap è un array che può essere visto come un albero binario quasi completo, quindi tutti i livelli saranno occupati, con gli elementi inseriti da sinistra verso destra, ma l'ultimo livello potrebbe non essere riempito totalmente. Lo heap, oltre ad avere la proprietà *length* appartenente anche agli array, dispone anche della proprietà *heapsize* che indica la quantità di elementi che fanno parte dello heap e di quanti non ne fanno parte, quindi $heapsize(A) \leq Length(A)$.

La radice è l'elemento più grande della struttura, quindi $A[Parent(i)] \geq A[i]$, essendo lo heap un array è possibile lavorare con gli indici, il parent di un nodo corrisponde a $floor(\frac{i}{2})$, il sottoalbero sinistro è $2i$ mentre quello destro $2i + 1$. Lo heap è ben bilanciato pertanto la sua altezza corrisponde a $log(n)$

Heapify:

Gli heap hanno un'operazione chiamata heapify che consente di ottenere un heap trasformato in modo che abbia l'elemento maggiore dell'array come radice, e tutti i numeri nei sottoalberi ordinati allo stesso modo. Questo avviene identificando il più grande elemento tra l'elemento in input e i suoi figli, e nel caso non sia il più grande quello in input, scambiare il valore dell'input ed il più grande e richiamare la procedura sul figlio corrispondente, in modo da ordinare ricorsivamente tutti gli elementi dello heap scambiati.

Heap sort:

L'algoritmo di ordinamento heap sort si basa sul costruire un heap con l'array di elementi in input, e a ripetizione scambiare la radice dello heap (quindi la testa all'array) con l'elemento in ultima posizione (quindi la coda all'array), poi chiamare la procedura heapify per riordinare lo heap, ed escludere l'ultimo elemento decrementando lo heapsize.